



IT Audit of DevOps

Issues Summary

05 August 2025

Observations:

- 1. Absence of a Formal DevOps Policy Document GOVERNANCE
- 2. DevOps Risk Assessment Not Performed Across People, Process, and Technology Dimensions SECURITY
- 3. Absence of Minimum Baseline Security Standards (MBSS) for DevOps Practices SECURITY
- 4. DevOps Roles and Responsibilities Not Aligned with the Organizational Manual IAM
- 5. Unapproved and Unaligned DevOps Process Documentation GOVERNANCE
- 6. Personal Access Token (PAT) Restrictions Not Enforced in Azure DevOps
- 7. Secrets Not Properly Protected in Azure DevOps Pipeline Variables APM
- 8. Azure Key Vault Not Integrated with Production Pipelines for Secrets Management apm
- 9. Absence of Integrated Vulnerability Scanning and Remediation Workflows in CI/CD Pipelines APM calls security
- 10. Hardcoded Secret in Source Code Repository
- 11. Uncontrolled Direct Deployments to Production APM
- 12. Secret Scanning not Implemented
- 13. Lack of Container Image Vulnerability Scanning and Ownership Security
- 14. Insecure Terraform Code Configurations and Absence of Pre-Deployment Review APM + Cloud
- 15. Absence of Workload Hardening and Runtime Threat Detection in Kubernetes Cluster Security
- 16. AKS Cluster Is Publicly Exposed, Over-Privileged, and Lacking Network Segmentation cloud n infra
- 17. Azure Monitor Lacks Defined Ownership and Operational Accountability TOOLS
- 18. Lack of Operational Ownership and Threat Visibility Despite DevOps Audit Logging Being Enabled security

Issue 1: Absence of a Formal DevOps Policy/Framework Document

A formal **DevOps policy document** is a cornerstone for establishing structured, secure, and scalable software delivery practices. It bridges strategic governance with operational execution by **translating identified risks into enforceable controls, policies**, and standard operating procedures (SOPs). In mature environments, this progression begins with a DevOps-specific risk assessment, followed by defining a control environment, creating policy directives, and finally implementing process documentation. The policy ensures that DevOps activities—across build, test, deploy, and operate phases—are aligned with organizational goals, security standards, and regulatory obligations.

Organizations should define and enforce a DevOps Policy that:

- Is built upon a foundational DevOps risk assessment that identifies threats across people, process, and technology layers
- Maps those risks to an appropriate control environment (e.g., RBAC, policy-as-code, artifact scanning, release gating)
- Codifies these controls into a formal policy document that outlines roles, responsibilities, tool governance, and security practices
- Aligns with frameworks such as ISO/IEC/IEEE 32675:2022, NIST SP 800-218, and internal audit control requirements
- Serves as the governing reference for all downstream DevOps SOPs (e.g., CI/CD workflows, secrets management, access provisioning, vulnerability handling)
- Is reviewed periodically and updated based on changes in architecture, threat landscape, tooling, and compliance needs

Description:

Internal Audit noted that IsDB lacks a formal, approved DevOps policy/framework document. There is no structured DevOps risk assessment to identify potential threats or inefficiencies across the SDLC. As a result, a **control environment has not been defined, and no policy has been drafted to institutionalize governance expectations**. DevOps-related activities are carried out without a unified policy/framework—leaving roles undefined, security practices inconsistent, and toolchain management. The absence of a policy results in unstandardized practices across teams, limited visibility into control adherence, and reduced ability to enforce compliance and mitigating risks. This also hinders the development of downstream SOPs or configuration baselines, ultimately **weakening the IsDB's DevSecOps posture and increasing the risk of misconfigurations and security weaknesses**.

Issue 2: DevOps Risk Assessment Not Performed Across People, Process, and Technology Dimensions

A structured DevOps risk assessment serves as the foundation for establishing a robust control environment across the software development and deployment lifecycle. It enables organizations to identify vulnerabilities and threats related to people (e.g., roles, skills, accountability), processes (e.g., approvals, governance, change workflows), and technology (e.g., CI/CD pipelines, IaC, secrets management). Without such assessment, organizations lack the visibility needed to design appropriate controls, formulate relevant policies, and operationalize consistent and secure DevOps practices.

Organizations should conduct periodic DevOps risk assessments that:

- Identify and classify risks across the DevOps lifecycle, including tools, environments, and interfaces
- Evaluate likelihood and impact across security, operational, and compliance domains
- Define mitigating controls aligned to each risk category
- Inform the creation of DevOps-specific security policies and Minimum Baseline Security Standards (MBSS)
- Enable development of standard operating procedures (SOPs) and process documentation based on defined controls
- Involve stakeholders from Development, Operations, and Security teams
- Are reviewed and updated based on changes to architecture, toolchains, regulatory requirements, or threat landscape

Description:

Internal Audit observed that the IsDB has not conducted DevOps-specific risk assessment to identify all the risks related to the DevOps process and technology. There is no evidence of a documented risk register, control mapping, or evaluation of threats associated with multiple areas within the DevOps such as CI/CD pipelines, infrastructure automation, deployment tools, or third-party integrations. During process understanding meetings, it was confirmed that no formal risk assessment is performed when onboarding new tools, modifying deployment flows, or updating access privileges across the DevOps environment. This absence of risk identification has prevented the establishment of structured control requirements, which in turn has hindered the formulation of targeted policies and implementation of SOPs for secure and reliable DevOps operations. As a result, the current DevOps practice lacks defensibility, increases the risk of toolchain misconfigurations, secrets exposure, access mismanagement, and ultimately reduces the IsDB's ability to enforce proactive and consistent DevSecOps controls.

Issue 3: Absence of Minimum Baseline Security Standards (MBSS) for DevOps Practices

Minimum Baseline Security Standards (MBSS) are essential to institutionalize secure-by-design principles across the DevOps lifecycle. They define a uniform set of foundational security controls to be implemented across CI/CD pipelines, Infrastructure-as-Code (IaC), toolchains, and runtime environments. An MBSS provides teams with clear expectations, eliminates ambiguity in security practices, and enables consistent enforcement of security measures such as secrets management, access control, vulnerability scanning, and secure configurations. MBSS alignment with international frameworks like ISO/IEC/IEEE 32675:2022, ISO 27001, and NIST 800-53 is also critical for meeting audit and compliance obligations.

Organizations should define and maintain a centralized DevOps MBSS document that:

- Specifies mandatory security controls for each stage of the DevOps lifecycle
- Provides configuration baselines for CI/CD platforms, IaC templates, cloud environments, and deployment tools
- Incorporates secure defaults for common resources (e.g., no hardcoded secrets, restricted ports, RBAC enforcement)
- References industry standards such as CIS Benchmarks, OWASP, NIST, and internal control frameworks
- Requires periodic review and updates based on threat intelligence, regulatory changes, or toolchain evolution
- Integrates with Policy-as-Code and CI/CD pipelines for automated enforcement

Description:

Internal Audit noted the absence of a formally defined or approved Minimum Baseline Security Standard (MBSS) for DevOps. No enterprise-level documentation exists outlining minimum security controls or configuration baselines to be enforced across pipelines, infrastructure, and toolsets. During the understanding meetings interviews with key personnel from the Solution Delivery and Infrastructure Operations teams revealed that they were unaware of any such requirements and do not reference established benchmarks (e.g., CIS Benchmarks) when configuring environments or integrating tools. Security controls are applied inconsistently and are often based on individual discretion, rather than policy or standards. There is no checklist or governance layer enforcing critical practices such as artifact integrity validation, policy-as-code enforcement, or threshold-based vulnerability scans. This significantly increases the risk of misconfigurations, insecure deployments, privilege misuse, secrets exposure, and non-compliance with security frameworks. The lack of a defined MBSS also hinders automation of security gates and limits the maturity of DevSecOps implementation.

Issue 4: DevOps Roles and Responsibilities Not Aligned with the Organizational Manual

Clearly defined and approved roles and responsibilities are foundational for effective DevOps operations. They ensure that accountability is assigned across the lifecycle—from code commits to deployment, security validation, change approvals, and incident handling. In organizations like IsDB, the *Organizational Manual* defines functional mandates for IMDT departments such as IT Strategy & Governance, Technology Operations, and Solution Delivery. DevOps roles must be mapped accordingly to institutionalize accountability, streamline cross-functional collaboration, and maintain compliance with internal policies and audit standards.

DevOps-related roles and responsibilities should be:

- Formally documented and approved, in alignment with the enterprise's *Organizational Manual*
- Mapped to function-specific mandates across IMDT divisions (e.g., Solution Delivery, Technology Operations, Security Operations)
- Integrated within the DevOps policy and referenced in supporting process documents
- Tailored to critical activities such as secure coding, CI/CD configuration, vulnerability remediation, infrastructure provisioning, deployment approvals, and incident management
- Reflected in team charters, onboarding guides, and approval workflows, ensuring traceability and reducing ambiguity
- Aligned with internal controls to support audit readiness and enable consistent security enforcement

Description:

Internal Audit found that the organization has not formally documented DevOps-specific roles and responsibilities that align with the *Organizational Manual*. The existing operating model lacks clarity on role boundaries, accountability, and functional ownership of key activities.

During interviews with stakeholders from the Solution Delivery, Technology Operations, and Security Operations teams, the following was observed:

- **Unclear accountability** for critical DevOps activities such as vulnerability remediation, infrastructure-as-code (IaC) validation, secure code reviews, pipeline approval mechanisms, and secrets scanning.
- Overlapping duties between DevOps engineers, security analysts, and operations personnel—without formal RACI (Responsible, Accountable, Consulted, Informed) matrices.

- Missing alignment with IMDT's governance mandates—e.g., no linkage between release activities and Technology Operations' change management responsibilities, or between secure deployment and Security Operations' oversight.

Issue 5: Unapproved and Unaligned DevOps Process Documentation

DevOps process documentation forms the operational backbone for consistent and secure software delivery. In mature organizations, process documents are downstream deliverables created only after foundational elements—such as risk assessments, control environment definitions, and policy frameworks—are formally established. These documents operationalize the intent of the DevOps policy and provide actionable, auditable procedures that align development, operations, and security teams across the CI/CD lifecycle.

DevOps process documentation should:

- Be derived from a formal DevOps policy, which itself is based on a documented DevOps risk assessment and corresponding control environment
- Be formally approved by designated authorities (e.g., CIO, CISO, or DevOps Governance Council)
- Contain clear ownership, version control, effective date, and periodic review cycles
- Be mapped to internal control libraries, enterprise governance frameworks, and industry standards (e.g., ISO/IEC/IEEE 32675:2022, ISO 27001, NIST SP 800-218)
- Include structured DevSecOps checkpoints such as code review gates, secrets management, vulnerability scanning, and approval workflows
- Serve as a reference document for audits, tool onboarding, security validation, and change management across DevOps activities

Description:

Internal Audit reviewed the document titled “IMDT–DevOps Management Process–V2.0”, which was found to be unapproved, outdated, and misaligned with any foundational governance layer. The document lacks essential metadata such as document owner, approval signatures, version control, or effective review date.



IMDT-DevOps
Management Proces

Critically, it does not reference:

- Any formal DevOps risk assessment
- An approved DevOps policy document
- Defined control expectations across security, compliance, or quality

- Regulatory or internal frameworks like ISO/IEC/IEEE 32675:2022 or the enterprise control library adoption.

Additionally, key process components are missing or underdeveloped, including:

- DevSecOps enforcement mechanisms (e.g., threat modeling, secure coding practices, secrets scanning)
- Security gates or quality checks in the CI/CD pipeline
- Mapping roles and responsibilities tied to tool usage and workflow stages

The absence of an underlying control environment and policy makes this document procedurally weak, and its contents lack traceability and enforceability. As a result, the process documentation does not meet enterprise governance expectations, increases the likelihood of unfragmented DevOps practices, and creates risks of audit non-compliance, tools usage, security oversights, and accountability gaps.

Issue 6: Personal Access Token (PAT) Restrictions Not Enforced in Azure DevOps

Personal Access Tokens (PATs) are used in Azure DevOps to authenticate scripts and automation processes. Without proper controls, PATs can become a major security liability, especially in CI/CD environments, as they grant access to source code, pipelines, artifacts, and other sensitive services.

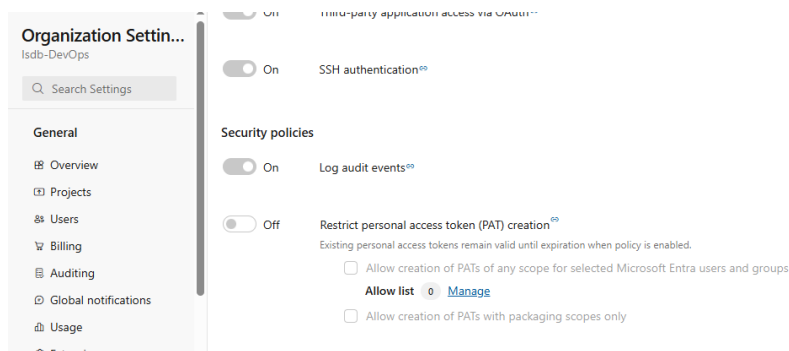
Organizations should enforce strict policies for PAT management to reduce the risk of credential misuse and unauthorized access. Best practices include:

- Enabling restrictions on PAT creation across the organization.
- Enforcing token scoping, mandatory expiration periods, and least-privilege permissions.
- Limiting PAT usage to specific trusted users or service accounts.
- Enabling audit logging to track creation and usage of PATs.
- Mandating Multi-Factor Authentication (MFA) for token issuance and renewal.

Description:

Internal Audit noted that the organization's Azure DevOps environment does not restrict PAT creation via organizational security policies. Specifically:

- The "Restrict personal access token (PAT) creation" setting is disabled.
- Any user can create PATs with broad scopes and indefinite lifespans.
- No allow-list or scope-based restriction is configured.



Discussions with the Infrastructure and Solution Delivery teams revealed that PATs are occasionally used in CI/CD pipelines or local scripts without centralized monitoring or secure vaulting.

This configuration significantly increases the risk of token leakage, abuse, and privilege escalation, especially if tokens are stored in plaintext, committed to source code repositories, or remain active long after their intended use.

Issue 7: Secrets Not Properly Protected in Azure DevOps Pipeline Variables

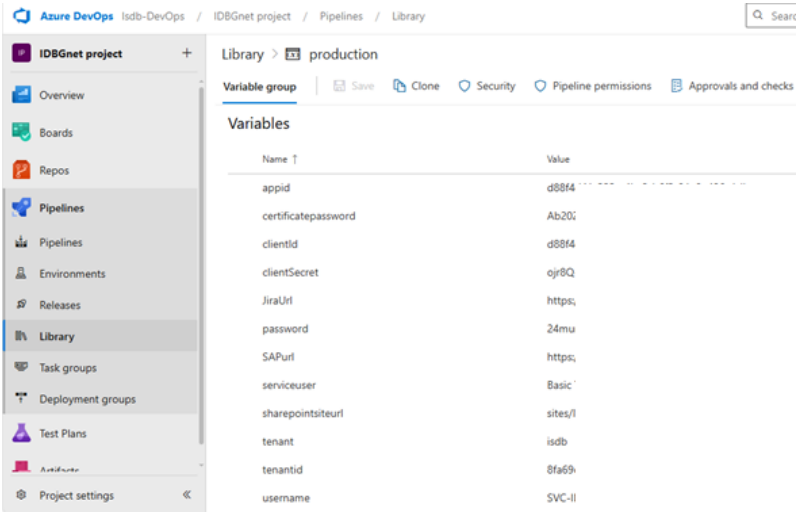
In secure DevOps environments, sensitive information such as credentials, client secrets, and access tokens must be stored and processed using secret-masking features. Azure DevOps offers a built-in capability to mark pipeline variables as "secrets", ensuring their values are encrypted, masked in logs, and not exposed to unauthorized users. Failing to enforce this control can lead to inadvertent data leaks and non-compliance with data protection regulations.

Sensitive variables should always be:

- Marked as secret when defined in Azure DevOps pipeline configurations.
- Masked from logs and hidden from users without elevated permissions.
- Protected from unauthorized read-access through access control mechanisms.
- Rotated regularly and stored in secure vaults where applicable (e.g., Azure Key Vault integration).
- Audited and validated during pipeline reviews and code deployments.

Description:

In the IDBGnet Production environment, Internal Audit observed that multiple sensitive variables, including: CertificatePassword, ClientSecret, Password, Username, TenantId, PPID, ClientId are not marked as secret and exposed in the Azure DevOps variable group settings.



Specifically:

- The "Keep this value secret" checkbox was left unchecked during variable creation.
- These values are visible in plaintext to any user with read access to the pipeline.
- Azure DevOps does not mask these values in pipeline logs, increasing the likelihood of accidental exposure during builds or script failures.

This violates secure coding and DevSecOps practices and exposes the organization to credential leakage and data breach.

Issue 8: Azure Key Vault Not Integrated with Production Pipelines for Secrets Management

Azure DevOps supports secure integration with Azure Key Vault, enabling the automated retrieval of secrets (e.g., API keys, credentials, certificates) during pipeline execution. This ensures that secrets are stored securely, rotated centrally, and never hardcoded or exposed within the DevOps environment. Integrating Azure Key Vault into CI/CD is considered a foundational DevSecOps practice that enhances both security and compliance.

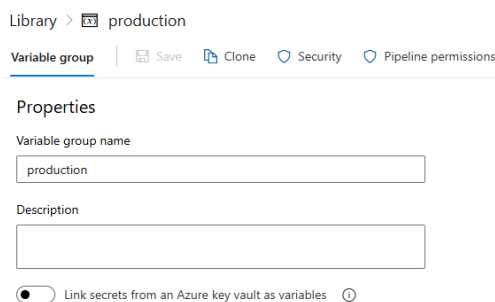
In a secure production environment:

- The “Link secrets from an Azure Key Vault as variables” option should be enabled.
- Pipelines should dynamically fetch secrets during runtime from Azure Key Vault, ensuring:
 - Centralized governance and audit logging.
 - Seamless rotation of credentials without manual intervention.
 - Elimination of hardcoded or environment-specific secrets from pipelines.
- Access to the vault should be controlled through RBAC and monitored for misuse or anomalies.

Description:

During the review of the IDBGnet Production environment, it was noted that:

- The toggle for “Link secrets from an Azure Key Vault as variables” was disabled in the production variable group.



- As a result:
 - Secrets are manually stored within the pipeline environment, often as plaintext variables.
 - There is no centralized control, versioning, or rotation mechanism for sensitive credentials used during deployment.
 - Security risks are significantly elevated, as credentials could be:
 - Exposed in logs,
 - Accidentally committed to source code repositories,
 - Read or copied by users with basic access to the variable group.

This deviates from DevSecOps best practices and increases the organization's vulnerability to credential misuse, supply chain compromise, and non-compliance with frameworks and international standards such as ISO 27001 (A.10.1.1), NIST SP 800-53 (SC-12, SC-28), and Microsoft Azure's security guidance.

Issue 9: Absence of Integrated Vulnerability Scanning and Remediation Workflows in CI/CD Pipelines

In modern DevOps environments, integrating security scans directly into CI/CD pipelines is a foundational aspect of DevSecOps. This includes static application security testing (SAST), dynamic application security testing (DAST), container scanning, and dependency vulnerability analysis. Automated scanning ensures that vulnerabilities are identified early in the software development lifecycle and that builds can be halted if critical issues are detected. Additionally, integrating issue trackers like Jira facilitates seamless remediation tracking and closure.

A mature DevOps environment should:

- Integrate security scanning tools (e.g., SonarQube, Snyk, Checkov, Aqua, WhiteSource) within build and release pipelines.
- Define logic to fail or block pipelines based on vulnerability severity thresholds.
- Maintain vulnerability remediation SLAs and dashboards for visibility.
- Integrate with tools like Jira or Azure Boards to auto-create tickets for identified issues and track closure.
- Generate audit logs and reports for traceability and compliance purposes.

Description:

IsDB has not implemented any vulnerability scanning tools within its Azure DevOps pipelines. Based on system walkthroughs and discussions with the Solution Delivery team:

- No security or vulnerability scanning is conducted during build, test, or release stages.
- Pipelines lack conditional logic to fail builds or block releases if vulnerabilities are identified.
- There are no SLAs, dashboards, or metrics tracking remediation timelines or severity trends.
- Jira is not integrated with Azure DevOps, resulting in no formal workflow for logging or assigning vulnerabilities (even if manually discovered).

As a result, the organization lacks visibility into vulnerabilities, is exposed to undetected risks in production deployments, and fails to meet security hygiene expectations.

Issue 10: Hardcoded Secret in Source Code Repository

Storing secrets directly in source code repositories—whether private or public—is a critical security risk. Hardcoded secrets such as API keys, passwords, and client secrets can be easily exposed through source control access, version history, or even accidental sharing. Industry standards like OWASP, NIST, and ISO 27001 strongly discourage this practice and recommend using secret managers such as Azure Key Vault, AWS Secrets Manager, or environment variables managed via CI/CD tooling.

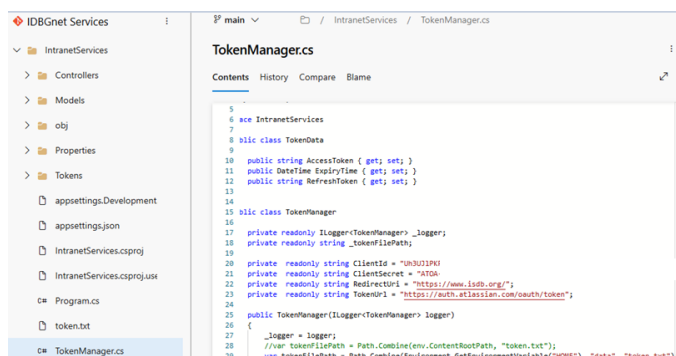
Organizations should:

- Prohibit hardcoding of secrets in application source code.
- Enforce use of secure secret storage (e.g., Azure Key Vault).
- Scan source code for exposed secrets using tools like GitGuardian, TruffleHog, or gitleaks.
- Integrate secret detection tools into the CI pipeline to fail builds on exposure.
- Implement security training and coding guidelines to reinforce secure practices.

Description:

A review of the IDBGnet services application codebase revealed that the file TokenManager.cs contains a hardcoded ClientSecret. This directly violates secure coding practices and introduces the following risks:

- The secret is stored in plaintext and is accessible to anyone with repository access.
- If committed, the secret becomes part of the permanent Git history, even if later removed.
- Exposure can occur via code screenshots, logs, backups, or automated tooling.
- There is no evidence of scanning tools or pre-commit hooks that could have flagged this security issue.



```
1 namespace IntranetServices
2 {
3     public class TokenData
4     {
5         public string AccessToken { get; set; }
6         public DateTime ExpiryTime { get; set; }
7         public string RefreshToken { get; set; }
8     }
9
10    public class TokenManager
11    {
12        private readonly ILogger<TokenManager> _logger;
13        private readonly string _tokenFilePath;
14
15        private readonly string ClientId = "UNSUBSCRIBED";
16        private readonly string ClientSecret = "AT0A";
17        private readonly string RedirectUrl = "https://www.idbg.org/";
18        private readonly string TokenUrl = "https://auth.atlassian.com/oauth/token";
19
20        public TokenManager(ILogger<TokenManager> logger)
21        {
22            _logger = logger;
23            //var tokenFilePath = Path.Combine(env.ContentRootPath, "token.txt");
24            var tokenFilePath = Path.Combine(Environment.GetEnvironmentVariable("HOME"), "data", "token.txt");
25        }
26    }
27 }
```

This practice significantly increases the risk of unauthorized access to connected services and compromises data confidentiality and compliance with controls such as ISO 27001 (Management of privileged access rights), OWASP ASVS – (Cryptographic storage and credential management).

Issue 11: Uncontrolled Direct Deployments to Production

Controlled and structured release management is a foundational principle of secure DevOps practices. Deployments should follow a defined path—from development to UAT and finally production—accompanied by appropriate validations, approvals, and rollback plans. This ensures code quality, minimizes the risk of production outages, and supports traceability and accountability.

Organizations should enforce:

- Stage-wise deployments (Dev → UAT → Production) with environment-specific validations.
- Pre-deployment approvals and quality gates, particularly before production releases.
- Automated controls to restrict direct production deployments without prior testing.
- Complete audit trails linking each release to its originating build, test results, and approval logs.

Description:

Review of the Azure DevOps release history - Sprint-eforms-ss0 release pipeline from IDBGnet project revealed multiple instances of direct deployments to the production environment, bypassing intermediate development and UAT stages.

Azure DevOps

Isdb-DevOps / IDBGnet project / Pipelines / Releases

Overview

Boards

Repos

Pipelines

Pipelines

Environments

Releases

Library

Task groups

Deployment groups

Test Plans

Artifacts

Connection with live updates has been terminated. Please refresh the page to see the updated view

Search all pipelines

+

New

Sprint38/4 all environment

Dev

Sprint58/7 all environment

uat

Sprint5 all environment

dev

Sprint18/2 all environment

dev

Sprint-eforms-ss0

production

IDBGnet project Header

production

IDBGnet Services all environme...

Production

Sprint-eforms-ss0

Releases

Deployments

Analytics

	Created	Stages
Release-179 20240716.2 dev	5/4/2025, 2:35:47 PM	dev uat production
Release-178 20240716.2 dev	1/16/2025, 12:37:59 PM	dev uat production
Release-177 20240716.2 dev	1/16/2025, 12:36:01 PM	dev uat production
Release-176 20240716.2 dev	7/16/2024, 12:41:38 PM	dev uat production
Release-174 20240715... main	7/15/2024, 3:21:53 PM	dev uat production
Release-155 20240612... main	6/12/2024, 7:19:33 AM	dev uat production
Release-13 20231206.4 dev	1/31/2024, 9:47:40 AM	dev uat production
Release-3 20231205.7 dev	12/5/2023, 2:34:11 PM	dev uat production
Release-2 20231205.6 dev	12/5/2023, 2:31:34 PM	dev uat production

Notably:

- Release IDs 174 and 155 were deployed directly to production without passing through lower environments.
- No manual intervention, blocking conditions, or approval workflows were configured to prevent or validate these deployments.

- The current pipeline design allows unrestricted deployment to production, which exposes the organization to the risk of introducing untested, unverified, or insecure code into the live environment.

Issue 12: Secret Scanning not Implemented

Secret scanning is a fundamental DevSecOps control designed to detect hardcoded credentials, API keys, tokens, and other sensitive values before they are committed to source code or introduced into CI/CD pipelines. Modern version control and CI/CD platforms offer native or integrated secret scanning capabilities to prevent inadvertent exposure and support proactive remediation.

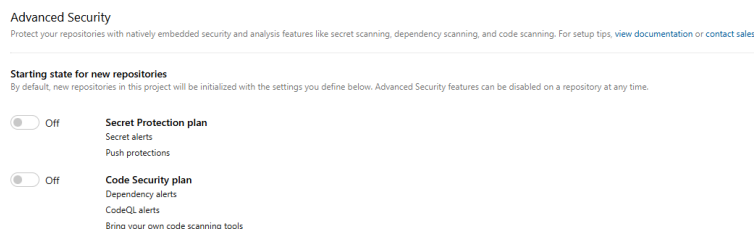
Organizations should implement automated secret scanning across all code repositories and pipelines by:

- Enabling push protection and pre-commit scanning to block commits with hardcoded secrets.
- Integrating secret scanning tools (e.g., GitHub Advanced Security, TruffleHog, Gitleaks) into CI/CD workflows.
- Creating a centralized alerting and remediation framework for discovered secrets.
- Defining SLAs for revocation, rotation, and impact analysis of exposed secrets.
- Enforcing secret handling policies in line with best practices and standards.

Description:

IsDB has not implemented any tooling or controls for secret scanning across its Azure DevOps environment:

- There is no scanning in place for secrets committed to source code repositories, introduced in pull requests, or used in CI/CD pipelines.
- Advanced Security settings confirm that both the Secret Protection Plan and Code Security Plan are disabled.



- There is no record of historical detections, no evidence of dashboards for monitoring, and no formal process in place to track, revoke, or rotate exposed credentials.
- As a result, hardcoded secrets such as API keys, passwords, and connection strings could exist undetected within the environment, posing a severe risk of credential leakage, and unauthorized access.

This constitutes a critical control gap, exposing IsDB to severe security and reputational risks, and misalignment with DevSecOps best practices and frameworks like ISO/IEC/IEEE 32675:2022, OWASP ASVS, and Microsoft DevSecOps Maturity Model.

Issue 13: Lack of Container Image Vulnerability Scanning and Ownership

In modern DevOps practices, containerized workloads form the backbone of scalable deployments. Securing these workloads begins with ensuring that container images are scanned for vulnerabilities before being pushed to production. Without automated image scanning and defined ownership, organizations face increased risks of deploying vulnerable components, misconfigured images, and non-compliance with security standards.

Organizations should enforce security controls around container image lifecycle by:

- Integrating vulnerability scanning tools (e.g., Azure Defender for Containers, Trivy, Aqua, or Twistlock) into the CI/CD pipelines and container registries.
- Blocking deployments of container images with high or critical vulnerabilities through policy-as-code or pipeline gates.
- Assigning ownership for container security to designated teams with clear roles and responsibilities.
- Tracking and remediating detected vulnerabilities within defined SLAs.
- Logging and auditing all image pushes, scans, and policy violations to ensure traceability and accountability.

Description:

During process understanding meetings, Internal Audit noted IsDB has significant control gaps in managing container image security:

1. There is no container image scanning tool integrated into the CI/CD pipelines or container registry.
2. Container images are built and deployed without any form of vulnerability scanning or policy enforcement.
3. Pipelines do not contain logic to fail builds or block releases based on vulnerability severity thresholds.
4. There is no assignment of ownership or formal responsibility for managing container security.
5. As a result, there is no implementation of scanning, monitoring, alerting, or compliance measures, leaving the environment vulnerable to known exploits and misconfigurations.

This lack of visibility and control introduces a high security risk of propagating vulnerable software into production environments, undermines incident readiness, and fails to meet expectations under frameworks such as OWASP Docker Top 10, NIST 800-190, and Azure Container Security Best Practices.

Issue 14: Insecure Terraform Code Configurations and Absence of Pre-Deployment Review

Infrastructure-as-Code (IaC) is a foundational component of DevOps that enables repeatable and scalable provisioning of cloud infrastructure. When poorly configured, IaC templates (such as Terraform) can introduce systemic security vulnerabilities across all environments. Ensuring secure-by-default templates and scanning mechanisms is critical for DevSecOps maturity, especially in cloud-native environments like Azure.

Organizations should establish robust IaC security governance by:

- Enforcing secure configuration baselines within Terraform modules (e.g., TLS 1.2, HTTPS-only, RBAC, IP whitelisting).
- Conducting automated static analysis of Terraform code using tools like Checkov, tfsec, or Terrascan during the CI phase.
- Applying role-based access control to Terraform state storage and disabling public access to backend blob containers.
- Defining clear ownership for IaC security reviews and ensuring that insecure configurations block deployments.

Description:

Internal Audit identified significant security and configuration gaps in Terraform-based infrastructure provisioning:

1. Terraform code is missing key security parameters across multiple Azure services:
 - Example: `https_only`, `min_tls_version`, `ssl_enforcement_enabled`, and `api_server_authorized_ip_ranges` are either absent or commented out in `.tf` files.
 - NSG rules were found to use wildcard "*" entries for sensitive ports (22, 3389), violating secure networking standards. The configuration parameters were missing.
2. Azure Key Vault configurations are partially secure, but `purge_protection_enabled` is inconsistently applied, creating risk during accidental or malicious deletions.
3. Azure Blob Storage used for remote Terraform state has not disabled public access, exposing the backend to potential data leaks.
4. There is no pre-deployment validation or enforcement mechanism.
5. No team has been assigned ownership of IaC security review or scanning, leading to governance breakdown.

Azure Service	Expected Secure Configuration	Observation
Azure Storage	min_tls_version = "TLS1_2"https_only = trueenable_https_traffic_only = truepublic_network_access_enabled = false	Missing in code
Azure Key Vault	soft_delete_enabled = truepurge_protection_enabled = trueLeast privilege access policies	Partially configured; missing purge protection in some areas
Azure SQL / PostgreSQL	public_network_access_enabled = falsessl_enforcement_enabled = true	Parameters missing
Azure App Services	https_only = trueclient_cert_enabled = true (if applicable)	Parameters missing
NSGs/Subnets	Avoid source_address_prefix = "*" on sensitive ports (22, 3389, 1433)	Parameters missing
Azure Kubernetes (AKS)	RBAC = true, api_server_authorized_ip_ranges, Azure AD integration	Security hardening configs missing

These configuration flaws collectively expose the organization to:

- Insecure deployments (e.g., public-facing DBs, weak TLS)
- Data exposure via open Terraform backend containers
- Compliance failures with frameworks like NIST 800-190, Azure CIS Benchmarks, and ISO/IEC/IEEE 32675:2022



ODM.zip

This organizationext: tf backend "azurerm"Q

Code5Work item0Wiki0Package0

Projects: AllRepo: AllBranch:Path:

Showing 5 code results

terraform.tfODM > dcp4 matches

/yemen/yemen-terraform/main/terraform.tf

terraform.tfODM > ODM4 matches

/terraform/container-registry/terraform.tf

terraform.tfODM > ODM4 matches

/terraform/infrastructure/terraform.tf

terraform.tfODM > ODM4 matches

/terraform/network/terraform.tf

terraform.tfODM > ODM4 matches

/terraform/app-registration/terraform.tf

terraform.tf

ContentsHistoryCompareBlame

```
1 terraform {
2   required_providers {
3     azuread = {
4       source = "hashicorp/azuread"
5       version = ">= 2.35"
6     }
7     azurerm = {
8       source = "hashicorp/azurerm"
9       version = ">= 3.40"
10    }
11    time = {
12      source = "hashicorp/time"
13    }
14  }
15  backend "azurerm" {
16    resource_group_name = "rg-terraform"
17    # storage_account_name = "isdbtfstore"
18    container_name = "terraform-state"
19    key = "app-registration-state"
20  }
}
```

compliance Control Name	recommendationSta	subscriptionId	subscriptionName	resourceGroup
Ensure guest users are reviewed on a monthly basis	Failed	99ac655a-cceb-485c-82b4-2de5785d85d5	isdb-odmprod-01	
Ensure that 'Public access level' is set to Private for blob containers	Failed	99ac655a-cceb-485c-82b4-2de5785d85d5	isdb-odmprod-01	rg-isdb-odm-prd-01
Ensure that 'Public access level' is set to Private for blob containers	Failed	99ac655a-cceb-485c-82b4-2de5785d85d5	isdb-odmprod-01	rg-isdb-odm-qas-01
Ensure that 'Public access level' is set to Private for blob containers	Failed	99ac655a-cceb-485c-82b4-2de5785d85d5	isdb-odmprod-01	rg-terraform
Ensure Default Network Access Rule for Storage Accounts is Set to Deny	Failed	99ac655a-cceb-485c-82b4-2de5785d85d5	isdb-odmprod-01	rg-isdb-odm-prd-01

MicrosoftDefender
ForCloudRegulatory

Issue 15: Absence of Workload Hardening and Runtime Threat Detection in Kubernetes Cluster

In containerized environments such as Azure Kubernetes Service (AKS), workload hardening and runtime threat detection are essential to reduce the attack surface and detect malicious behavior. Kubernetes offers native capabilities—like securityContext, Pod Security Admission (PSA), and integration with Azure Defender—to enforce container security baselines and monitor threats in real time.

Organizations operating AKS environments should:

- Enforce Kubernetes Pod Security Standards by defining restrictive securityContext settings (e.g., disallow running as root, disable privilege escalation).
- Apply Pod Security Admission (PSA) labels (e.g., enforce=restricted) across all namespaces to prevent insecure deployments.
- Integrate runtime threat detection tools, such as Microsoft Defender for Containers, Falco, or Aqua Security, to monitor suspicious behavior and container-level anomalies.
- Use hardened container registry tiers (e.g., Azure Premium ACR) to ensure build integrity, vulnerability scanning, and image trust.

Description:

- The current Kubernetes deployment manifests lack security hardening configurations:
 - securityContext is missing, allowing containers to run as root.
 - No restrictions such as allowPrivilegeEscalation: false or capabilities: drop are applied.

ODM / Search

This project

kind: Deployment ext: yaml

Q

Code 83

Wiki 0

Search this organization

Repo: All

Branch:

Path:

Showing 83 code results

financing-service-deployment.yaml

ODM > dcp

1 match

gcp-graphql-server-deployment.yaml

ODM > dcp

1 match

procurement-service-deployment.yaml

ODM > dcp

1 match

project-service-deployment.yaml

ODM > dcp

1 match

audit-service-deployment.yaml

ODM > dcp

1 match

email-service-deployment.yaml

ODM > dcp

1 match

file-service-deployment.yaml

ODM > dcp

1 match

form-service-deployment.yaml

ODM > dcp

1 match

identity-service-deployment.yaml

ODM > dcp

1 match

messaging-service-deployment.yaml

ODM > dcp

1 match

parameter-service-deployment.yaml

ODM > dcp

1 match

state-service-deployment.yaml

ODM > dcp

1 match

financing-service-deployment.yaml

Contents History Compare Blame

```
4 metadata:
5   name: gcp-financing-service
6   labels:
7     app: gcp-financing-service
8 spec:
9   type: ClusterIP
10  selector:
11    app: gcp-financing-service
12  ports:
13    - name: http
14      port: 80
15      targetPort: 8080
16
17 ---
18 apiVersion: apps/v1
19 kind: Deployment
20 metadata:
21   name: gcp-financing-service
22   labels:
23     app: gcp-financing-service
24 spec:
25   revisionHistoryLimit: 3
26   selector:
27     matchLabels:
28       app: gcp-financing-service
29   template:
30     metadata:
31       labels:
32         app: gcp-financing-service
33     annotations:
34       prometheus.io/scrape: "true"
35       prometheus.io/port: "8080"
36       prometheus.io/path: "/prometheus"
37       prometheus.io/scheme: "http"
38   spec:
39     containers:
40       - name: gcp-financing-service
41         image: #(image)#
42         imagePullPolicy: Always
43         ports:
44           - name: http
45             containerPort: 8080
46             protocol: TCP
47
```

Isdb-DevOps / ODM / Search

This project

kind: Deployment ext: yaml securityContext

Q

Code 0

Wiki 0

Search this organization

Repo: All

Branch:

Path:

Filters (e.g., Activity exts)

ext: With file extension

file: Filename

path: Under path

pkg: Inside project

repo: Inside repository

basetype: Basetype

class: Class

comment: Comment

decl: Declaration

def: Definition

Show more

Operators (e.g., To-Do OR re-list)

AND NOT OR

For more ways to search, see the help page

No code files found for 'kind: Deployment ext:...' with applied filters

- Pod Security Admission (PSA) enforcement is disabled
 - Labels such as pod-security.kubernetes.io/enforce=restricted are not present.
 - There are no warn or audit settings, meaning insecure workloads are neither rejected nor logged.

FAVOURITMacBook-Pro Downloads % kubectl get ns --show-labels				
NAME	STATUS	AGE	LABELS	
ols-command	Active	11m	kubernetes.io/metadata.name=ols-command	
app-routing-system	Active	534d	kubernetes.io/metadata.name=app-routing-system	
cg	Active	534d	kubernetes.io/metadata.name=cg	
default	Active	534d	kubernetes.io/metadata.name=default	
gatekeeper-system	Active	534d	addonmanager.kubernetes.io/mode=Reconcile,admission.gatekeeper.sh/ignore-no-self-managing-control-plane=controller-manager.gatekeeper.sh/system=yes,kubernetes.azure.com/managedby=aks,kubernetes.io/metadata.name=gatekeeper-system	
gcp	Active	534d	kubernetes.io/metadata.name=gcp	
health	Active	534d	kubernetes.io/metadata.name=health	
kube-node-lease	Active	534d	kubernetes.io/metadata.name=kube-node-lease	
kube-public	Active	534d	kubernetes.io/metadata.name=kube-public	
kube-system	Active	534d	addonmanager.kubernetes.io/mode=Reconcile,control-plane=true,kubernetes.azure.com/managedby=aks,kubernetes.io/cluster-service=true,kubernetes.io/metadata.name=kube-system	
monitoring	Active	525d	kubernetes.io/metadata.name=monitoring	
odm	Active	534d	kubernetes.io/metadata.name=odm	
pg	Active	121d	kubernetes.io/metadata.name=pg	

```
AVOUR@MacBook-Pro Downloads % az security pricing show --name ContainerRegistry
/opt/homebrew/Cellar/azure-cli/2.75.0/libexec/lib/python3.12/site-packages/azure/mgmt/security/v2023_01_01/models/_models_py3.py:102: SyntaxWarning: invalid escape sequence '\
Available for CloudPosture plan.:code: <br>':code: <br>'\ **OnUploadMalwareScanning** - Limits
/opt/homebrew/Cellar/azure-cli/2.75.0/libexec/lib/python3.12/site-packages/azure/mgmt/security/v2023_01_01/models/_models_py3.py:150: SyntaxWarning: invalid escape sequence '\
Available for CloudPosture plan.:code: <br>':code: <br>'\ **OnUploadMalwareScanning** - Limits
/opt/homebrew/Cellar/azure-cli/2.75.0/libexec/lib/python3.12/site-packages/azure/mgmt/security/v2023_01_01/models/_models_py3.py:259: SyntaxWarning: invalid escape sequence '\
the date of the last time the "pricingTier" was set to "Standard"\, when available (e.g
{
  "deprecated": true,
  "enablementTime": null,
  "extensions": null,
  "freeTrialRemainingTime": "30 days, 0:00:00",
  "id": "/subscriptions/99ac655a-cc6b-485c-82b4-2de5785d85d5/providers/Microsoft.Security/pricings/ContainerRegistry",
  "name": "ContainerRegistry",
  "pricingTier": "Free",
  "replacedBy": [
    "Containers"
  ],
  "subPlan": null,
  "type": "Microsoft.Security/pricings"
```

Issue 16: AKS Cluster configuration weaknesses

Security in Kubernetes environments relies on three foundational pillars: identity governance, network segmentation, and controlled access to the control plane. Misconfigurations in these areas significantly expand the attack surface and weaken overall cluster resilience. In regulated and enterprise environments, these misconfigurations can lead to security breaches, data exposure, and regulatory non-compliance.

An enterprise-grade AKS (Azure Kubernetes Service) environment should:

- Grant cluster-admin privileges strictly through controlled Azure AD groups with proper RBAC enforcement, not directly to individual user accounts.
- Enforce namespace-level microsegmentation using NetworkPolicy, including default-deny rules and explicit allow-lists for communication between pods and namespaces.
- Configure the AKS cluster as private, ensuring the Kubernetes API server is accessible only via trusted networks or private endpoints.

Description:

1. Excessive Privilege Assignment:

- A ClusterRoleBinding named aks-cluster-admin-binding grants the cluster-admin role directly to individual users: clusterAdmin and clusterUser.
- These users are not part of any AD group, meaning there's no centralized identity governance or access lifecycle control.
- This reduces auditability, circumvents principle of least privilege, and creates a risk of unauthorized administrative access.

```
FAVOUR@MacBook-Pro Downloads % kubectl describe clusterrolebinding aks-cluster-admin-binding
Name:          aks-cluster-admin-binding
Labels:        addonmanager.kubernetes.io/mode=Reconcile
               kubernetes.io/cluster-service=true
Annotations:   <none>
Role:
  Kind: ClusterRole
  Name: cluster-admin
Subjects:
  Kind  Name      Namespace
  ----  ---      -
  User   clusterAdmin
  User   clusterUser
FAVOUR@MacBook-Pro Downloads % kubectl get clusterroles -o yaml > clusterroles.yaml
FAVOUR@MacBook-Pro Downloads % kubectl auth can-i get pods --as clusterUser -n cg
yes
FAVOUR@MacBook-Pro Downloads %
```

2. Lack of Network Segmentation:

- The cluster contains only one NetworkPolicy, applied to the kube-system namespace for a system component (konnectivity-agent).
- No default-deny policies exist in operational namespaces (default, production, etc.), allowing unrestricted pod-to-pod communication.
- This violates zero-trust and microsegmentation principles, increasing the risk of lateral movement and east-west attacks in case of container compromise.

```
FAVOUR@MacBook-Pro Downloads % kubectl get networkpolicies -n cg
No resources found in cg namespace.
FAVOUR@MacBook-Pro Downloads % kubectl get networkpolicies -A
NAMESPACE      NAME                      POD-SELECTOR          AGE
kube-system     konnectivity-agent       app=konnectivity-agent 534d
```

3. Publicly Exposed AKS Cluster:

- The AKS cluster is not configured as private.
- The apiServerAccessProfile field is absent from the configuration, confirming that the Kubernetes API server is publicly accessible.
- Combined with the above privilege and network gaps, this significantly elevates the risk of external exploitation, brute-force attempts, or abuse of public endpoints.

```
FAVOUR@MacBook-Pro Downloads % az aks show --resource-group rg-isdb-odm-prd-01 --name aks-isdb-odm-prd-01 --query apiServerAccessProfile
FAVOUR@MacBook-Pro Downloads %
```

Issue 17: Azure Monitor Is Configured but Lacks Defined Ownership and Operational Accountability

In containerized environments like AKS, observability is a critical pillar of both performance monitoring and security assurance. Tools such as Azure Monitor and Log Analytics provide the capability to collect telemetry from containers, nodes, and workloads. However, without defined ownership and operational accountability, the mere presence of monitoring tools does not translate into actionable insight or response readiness.

An effective monitoring and observability setup should:

- Assign clear ownership for monitoring dashboards, alerts, and incident response workflows.
- Include Standard Operating Procedures (SOPs) for triaging, investigating, and escalating incidents derived from telemetry data.
- Have defined KPIs/SLAs for monitoring coverage, alert response times, and log review frequency.
- Ensure alerts are integrated with notification systems (e.g., Teams, PagerDuty) and mapped to operational workflows.

Description:

- The AKS cluster is configured with Azure Monitor (omsagent), and telemetry data is correctly routed to Log Analytics Workspace (log-isdb-workspace-01).
- Metrics and logs from AKS nodes, pods, and container runtimes are available and accessible, indicating that observability tooling is technically operational.

```
FAVOUR@MacBook-Pro Downloads % az aks show --resource-group rg-isdb-adr-prd-01 --name aks-isdb-adr-prd-01 --query omsagentProfiles
{
  "azureKeyVaultSecretsProvider": {
    "config": {
      "enableSecretRotation": "true",
      "rotationPollInterval": "2m"
    },
    "enabled": true,
    "identity": {
      "clientId": "8ef879b2-3b8a-4a2f-929e-1c6178a2ac22",
      "objectId": "e65601a9-6ef9-4b7b-b0ce-6acca049cf25",
      "resourceId": "/subscriptions/990c655a-ccb-485c-82b4-2de5785d85d5/resourcegroups/rg-isdb-adr-aks-node-prd-01/providers/Microsoft.ManagedIdentity/userAssignedIdentities/azurekeyvaultsecretsprovider-aks-isdb-adr-prd-01"
    }
  },
  "azurpolicy": {
    "config": {
      "version": "v2"
    },
    "enabled": true,
    "identity": {
      "clientId": "d1b10e9b-d2f5-41b7-8166-8c99507ecf73",
      "objectId": "8b6827f6-3165-447b-9f56-a268f997ec10",
      "resourceId": "/subscriptions/990c655a-ccb-485c-82b4-2de5785d85d5/resourcegroups/rg-isdb-adr-aks-node-prd-01/providers/Microsoft.ManagedIdentity/userAssignedIdentities/azurpolicy-aks-isdb-adr-prd-01"
    }
  },
  "omsagent": {
    "config": {
      "logAnalyticsWorkspaceResourceId": "/subscriptions/990c655a-ccb-485c-82b4-2de5785d85d5/resourcegroups/rg-isdb-shared-01/providers/Microsoft.OperationalInsights/workspaces/log-isdb-workspace-01",
      "useAADAuth": "false"
    },
    "enabled": true,
    "identity": {
      "clientId": "ac7a7585-5221-4516-97fc-0c8e9c63f2b3",
      "objectId": "a63b8b8e-27e3-47bb-b08e-0a1059a813b",
      "resourceId": "/subscriptions/990c655a-ccb-485c-82b4-2de5785d85d5/resourcegroups/rg-isdb-adr-aks-node-prd-01/providers/Microsoft.ManagedIdentity/userAssignedIdentities/omsagent-aks-isdb-adr-prd-01"
    }
  }
}
```



```
FAVOURE@MacBook-Pro Downloads % az aks show --resource-group rg-isdb-odm-prd-01 --name aks-isdb-odm-prd-01 --query addonProfiles.omsagent.config.logAnalyticsWorkspaceResourceID  
"/subscriptions/99ac655a-cceb-485c-82b4-2de5785d85d5/resourceGroups/rg-isdb-shared-01/providers/Microsoft.OperationalInsights/workspaces/log-isdb-workspace-01"  
FAVOURE@MacBook-Pro Downloads %
```

- However, during interviews with stakeholders from the Infrastructure and Solution Delivery teams:
 - No designated owner or team was identified for monitoring the data.
 - There are no formal SOPs, SLAs, or escalation workflows for reviewing or responding to incidents, errors, or anomalies surfaced through monitoring.
 - Alerts and dashboards (if any) are not actively maintained or reviewed, making detection of availability, performance, or security events unreliable.

Issue 18: Lack of Operational Ownership and Threat Visibility Despite DevOps Audit Logging Being Enabled

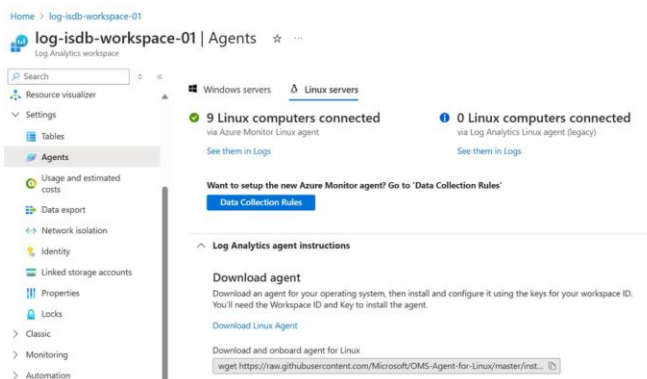
Effective DevOps governance depends not only on logging capabilities but also on operational ownership, real-time alerting, and contextual visibility into activities such as code changes, deployments, and security scan results. Pipeline and system logs offer crucial insights into potential misconfigurations, failed deployments, and unauthorized activity—but only when monitored, enriched, and correlated with other security systems.

Organizations should leverage their DevOps logging infrastructure by:

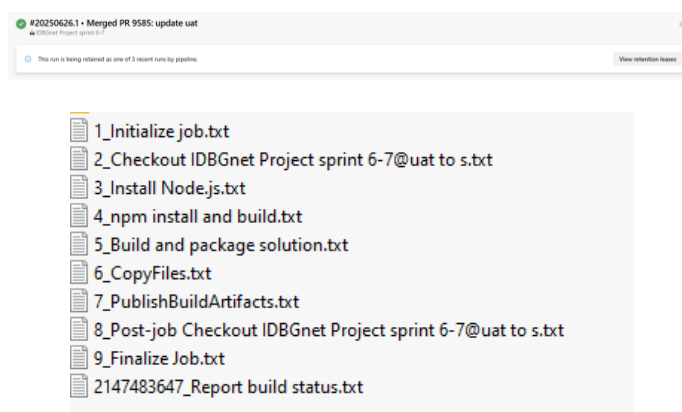
- Assigning clear ownership for monitoring and investigating pipeline and system-level logs.
- Integrating with SIEM platforms like Microsoft Sentinel for centralized correlation and alerting.
- Creating custom alerts and dashboards to detect:
 - Deployment anomalies
 - Policy bypasses
 - Secrets leakage or scan failures
- Using enriched logs with context tags (e.g., environment, criticality, sprint ID) for better triage and forensic traceability.

Description:

- Azure DevOps native auditing is enabled and integrated with Log Analytics Workspace (log-isdb-workspace-01), with a 1-year retention period.



- Pipeline logs for key deployments (e.g., PR #9585 in the IDBGnet Project) were reviewed and found to include detailed logs across all build and deployment stages:
 - Initialize job.txt
 - Checkout, npm install, Build and package, CopyFiles, PublishBuildArtifacts, and Report build status.txt



- However, the logging setup remains configured as default/vanilla and lacks enhancements required for actionable visibility.

During walkthroughs and interviews:

- No DevOps or InfoSec team has ownership of monitoring these logs.
- Logs are not integrated with Microsoft Sentinel or any SIEM.
- No custom alerts, log enrichment, or use-case-based dashboards have been implemented.
- Technology Operations team does not manage DevOps security logs, and Solution Delivery has not raised integration requests or set monitoring KPIs.